JAVASCRIPT - STRING LITERALS, OPERATORS AND CONDITIONAL STATEMENT

Strings & Template Literals in JavaScript

Strings are sequences of characters used to represent text. In JavaScript, they can be created in three ways:

Declaring Strings

1. **Single quotes (' ')**

   Ex: let str1 = 'Hello';

2. **Double quotes (" ")**

   Ex: let str2 = "World";

3. **Backticks (` `) (Template Literals)**

   Ex: let str3 = `Hello World`;

String Properties & Methods

- **length** → returns the number of characters.

  Ex:

  ```
  let text = "JavaScript";

  console.log(text.length); // 10
  ```

- `toUpperCase()` & `toLowerCase()` → change case.

  Ex:

  console.log(text.toUpperCase()); // JAVASCRIPT

  console.log(text.toLowerCase()); // javascript

- `slice(start, end)` → extracts part of a string.

  Ex:

  let text = "JavaScript";

  console.log(text.slice(0, 4)); // "Java" (from index 0 to 3)

- `includes()` → checks if substring exists.

  Ex:

  console.log(text.includes("Script")); // true

- `trim()` → removes whitespace.

  Ex:

  let msg = " Hi! ";

  console.log(msg.trim()); // "Hi!"

Concatenation using (+)

Joining strings with + operator.

Ex:

```
let firstName = "John";

let lastName = "Doe";

let fullName = firstName + " " + lastName;

console.log(fullName); // John Doe
```

Template Literals

Template literals allow embedding variables and expressions inside strings using ${ }.

Ex:

```
let name = "Alice";

let age = 25;

console.log(`Hello, my name is ${name} and I am ${age} years old.`);

// Hello, my name is Alice and I am 25 years old.
```

Multi-line Strings

Using backticks (``) you can write multi-line strings easily.

Ex:

```
let msg = `This is line 1

This is line 2

This is line 3`;

console.log(msg);
```

String Interpolation

Allows inserting expressions inside strings.

Ex:

```
let a = 5;

let b = 10;

console.log(`The sum of ${a} and ${b} is ${a + b}`);

// The sum of 5 and 10 is 15
```

Operators in JavaScript

Operators perform operations on values and variables.

1. Arithmetic Operators

Used for mathematical calculations.

Ex:

```
let x = 10, y = 3;

console.log(x + y); // 13 (Addition)

console.log(x - y); // 7 (Subtraction)

console.log(x * y); // 30 (Multiplication)

console.log(x / y); // 3.333... (Division)

console.log(x % y); // 1 (Remainder)
```

---

2. Assignment Operators

Used to assign values.

Ex:

```
let a = 5;

a += 2; // a = a + 2 → 7

a -= 2; // a = a - 2 → 5

a *= 2; // a = a * 2 → 10

a /= 2; // a = a / 2 → 5
```

3. Comparison Operators

Used to compare values, returns `true` or `false`.

Ex:

```
console.log(5 == "5");  // true  (checks value only)

console.log(5 === "5"); // false (checks value & type)

console.log(5 != "5");  // false

console.log(5 !== "5"); // true

console.log(10 > 5);    // true

console.log(10 <= 5);   // false
```

---

4. Logical Operators

Used for combining conditions.

Ex:

```
let age = 20;

console.log(age > 18 && age < 30); // true (AND)

console.log(age > 25 || age < 18); // false (OR)

console.log(!(age > 18));  // false (NOT)
```

---

5. Unary Operators

Operate on a single operand.

Ex:

```
let num = 5;

console.log(++num); // 6 (pre-increment)

console.log(num--); // 6 (then num becomes 5)

console.log(typeof num); // number
```

---

6. Ternary Operator

Shorthand for `if...else`.

Ex:

```
let marks = 75;

let result = (marks >= 50) ? "Pass" : "Fail";

console.log(result); // Pass
```

Conditional statement

A conditional statement in programming is a decision-making structure that allows a program to execute certain blocks of code only if a specific condition is true (or false).

1. if statement:

Executes a block of code only if the condition is true.

Ex:

```
let age = 20;

if (age >= 18) {

  console.log("You are eligible to vote.");

}
```

2. If-else

Provides an alternative block if the condition is false.

Ex:

```
let num = 5;

if (num % 2 === 0) {

  console.log("Even number");

} else {

  console.log("Odd number");

}
```

3. if-else if ladder

Checks multiple conditions in sequence.

Ex:

```
let marks = 75;

if (marks >= 90) {
  console.log("Grade: A+");
} else if (marks >= 75) {
  console.log("Grade: A");
} else if (marks >= 50) {
  console.log("Grade: B");
} else {
  console.log("Grade: C");
}
```

4. Switch statement

Used when there are many possible conditions for a single value.

Ex:

```
let day = 3;

switch (day) {
  case 1:
    console.log("Monday");
    break;
  case 2:
    console.log("Tuesday");
    break;
  case 3:
    console.log("Wednesday");
    break;
  case 4:
    console.log("Thursday");
    break;
  case 5:
    console.log("Friday");
    break;
  default:
    console.log("Weekend");
}
```

5. Nested conditionals

Condition inside another condition.

Ex:

```
let username = "admin";

let password = "12345";


if (username === "admin") {
  if (password === "12345") {
    console.log("Login successful!");
  } else {
    console.log("Wrong password!");
  }
} else {
  console.log("Invalid username!");
}
```

6. Short-circuiting with && and | |

- && executes the second statement only if the first is true.
- | | executes the second statement only if the first is false.

Ex:

```
let isLoggedIn = true;

let userName = "Maj";


// && → runs right side only if left is true

isLoggedIn && console.log("Welcome, " + userName);


// || → runs right side only if left is false

let displayName = userName || "Guest";

console.log("Hello, " + displayName);
```